

The maximum Laplacian ratio of a tree: a complete answer to a question of Brualdi and Goldwasser, kernel-checked in Lean

Peter W. Murphy, MD

26 September 2026 (preprint, not yet refereed)

Abstract

For a tree T with Laplacian $L(T)$, Brualdi and Goldwasser (1984) introduced the *Laplacian ratio* $\pi(T) = \text{per } L(T) / \prod_v \deg v$ and asked which tree on n vertices maximizes it. We answer the question for every n . For every $n \geq 4$ the maximum is attained by a *spider of cherry arms*: a centre carrying pendant paths of length two (cherries) and *arms*, an arm being a vertex that carries cherries. For $n \geq 492$ the centre carries only arms, almost all with five cherries, and the exceptional arms are fixed by $6(n-1) \bmod 11$ through an explicit rule; for $4 \leq n \leq 491$ the maximizer is given by an explicit table. Along the way we determine the exact exponential growth rate of the maximum, $\frac{64}{621}\rho^n \leq \max_{|T|=n} \pi(T) \leq 2\rho^{n-1}$ with $\rho = (621/64)^{1/11}$, and prove the sharp inequality behind it: every pendant subtree b has matching weight at most $\rho^{|b|}$, with equality exactly for the five-cherry arm. The whole argument is formalized in Lean 4 with Mathlib and checked by the Lean kernel, depending only on Lean’s three standard axioms. Large parts of it are certificates: exact data produced by untrusted generators and checked by the kernel. We explain the mathematics, show where each piece lives in the formal development and why it is organized that way, and discuss what the answer and the method suggest.

Contents

1	Introduction	2
1.1	The question	2
1.2	The answer	3
1.3	How the proof goes	4
1.4	How to read this paper	4
2	From permanents to matchings	5
3	The cavity recursion	6
4	The sharp ceiling and the growth rate	8
4.1	Subactions: paying for a vertex with its children’s credit	9
4.2	The growth rate	10
5	Every maximizer is a spider	10
5.1	Setting up the comparison	11
5.2	High degree: $k \geq 24$	11
5.3	Low degree and large n : linear deficits	13

5.4	Low degree, $150 \leq n \leq 491$: rate cells with credits	13
5.5	Low degree, $7 \leq n \leq 149$: the envelope certificate	14
5.6	The smallest trees, and the summary	14
6	The best spider	15
6.1	The closed form and the balance identity	15
6.2	Sizes up to 491: a kernel sweep	16
6.3	Sizes from 492: structure, then 223 certificates	16
7	What exactly the Lean theorem says	17
8	Certificates and Telperion	18
9	Independent computations	19
10	Discussion	20
10.1	What the answer says	20
10.2	Permanents	20
10.3	Method	20
10.4	What remains	21
11	Limitations	21
12	Reproducing the results	21
A	The maximizers for $4 \leq n \leq 491$	22

1 Introduction

1.1 The question

Let T be a tree on $n \geq 2$ vertices, D its diagonal degree matrix, A its adjacency matrix and $L(T) = D - A$ its Laplacian. Determinants of Laplacians are classical: by the matrix-tree theorem they count spanning trees. Permanents are a different matter. The permanent $\text{per } M = \sum_{\sigma} \prod_i M_{i,\sigma(i)}$ looks like the determinant without the signs, but it is $\#P$ -hard to compute in general, and there is no comparable structure theory.

Brualdi and Goldwasser [1] studied $\text{per } L(T)$ for trees and bipartite graphs. Since the permanent of a Laplacian grows with the degrees, they normalized it and introduced the *Laplacian ratio*

$$\pi(T) = \frac{\text{per } L(T)}{\prod_{v \in V(T)} \deg v}.$$

They showed that $\pi(T) \geq 2$ for every tree on at least two vertices, with equality exactly for the star, and they asked the natural companion question: *which tree on n vertices maximizes π ?*

The question stayed open for four decades. Wu, Dong and Lai [2] studied it again in 2025 and conjectured that the maximizer is a particular subdivided star. In 2026 Pant [3] refuted that conjecture with explicit families of caterpillars, paths whose vertices each carry many pendant paths of length two, and left the question open, proposing no replacement.

This paper answers it for every n .

1.2 The answer

The answer is built from two small pieces (Figure 1).

Definition 1 (cherries, arms, spiders). A *cherry* is a pendant path of length two. For $j \geq 1$ an *arm* A_j is a vertex, adjacent to the centre, that carries j cherries and nothing else; it has $2j + 1$ vertices. We write A_0 for a leaf adjacent to the centre. A *spider of cherry arms* (in this paper, simply a *spider*) is a tree with a vertex, the *centre*, all of whose branches are leaves, cherries or arms. We write $C^c A_{j_1} A_{j_2} \cdots$ for the spider whose centre carries c cherries and arms $A_{j_1}, A_{j_2}, \dots$, and use exponents for repeated arms.

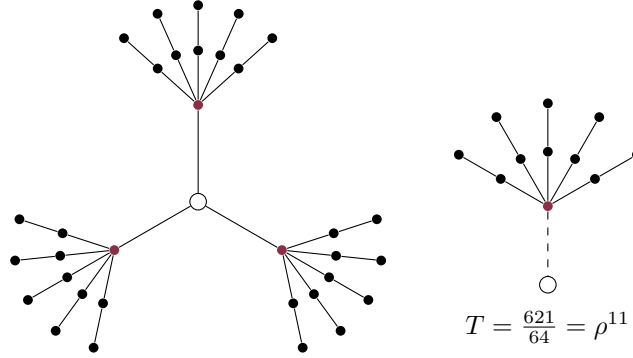


Figure 1: Left: the spider A_5^3 on 34 vertices (centre drawn open). For large n the maximizer looks like this, a centre carrying many five-cherry arms; small sizes are more varied, and at $n = 34$ itself the maximizer is $C^8 A_8$. Right: the five-cherry arm A_5 , an 11-vertex pendant subtree whose matching weight is exactly ρ^{11} . It is the unique pendant subtree attaining the sharp bound of Theorem 5.

Theorem 2 (main theorem). *For every $n \geq 4$ the tree $\text{bgMax}(n)$ below has n vertices and maximizes π over all trees on n vertices.*

1. *For $n \geq 492$, put $s = 6(n - 1) \bmod 11$. The centre of $\text{bgMax}(n)$ carries only arms, and all of them are A_5 except:*

- $s \in \{1, 2, 3, 4\}$: s arms are A_6 ; except that for $s = 3$ with $n \leq 722$ and for $s = 4$ with $n \leq 2319$, instead $11 - s$ arms are A_4 ;
- $s \in \{5, \dots, 10\}$: $11 - s$ arms are A_4 .

2. *For $4 \leq n \leq 491$, $\text{bgMax}(n)$ is the spider listed in Section A.*

For $n \leq 3$ there is only one tree on n vertices.

The recipe is easier to remember than it looks. An A_5 has 11 vertices, an A_4 has 9 and an A_6 has 13. After the centre there are $n - 1$ vertices to distribute, and $n - 1 \equiv 2s \pmod{11}$. Each A_6 used in place of an A_5 adds 2 modulo 11 and each A_4 subtracts 2, so s copies of A_6 , or $11 - s$ copies of A_4 , exactly absorb the remainder (Table 1). The theorem says which of the two ways wins. The two late switches at $n = 722$ and $n = 2319$ are genuine: they are where two very close competitors trade places (Figure 12).

$s = 6(n-1) \bmod 11$	0	1	2	3	4	5	6	7	8	9	10
non- A_5 arms until the switch	none	A_6	A_6^2	A_6^3 A_4^8 ($n \leq 722$)	A_6^4 A_4^7 ($n \leq 2319$)	A_4^6	A_4^5	A_4^4	A_4^3	A_4^2	A_4

Table 1: The rule of Theorem 2 for $n \geq 492$, class by class. All other arms are A_5 , and the centre carries no cherries or leaves.

Why five cherries. The number $\rho = (621/64)^{1/11} \approx 1.22948$ is the exact exponential growth rate of the maximum.

Theorem 3 (growth rate). *For every $n \geq 1$, $\frac{64}{621} \rho^n \leq \max_{|T|=n} \pi(T) \leq 2 \rho^{n-1}$.*

The upper bound comes from a sharp inequality for pendant subtrees (Theorem 5): every pendant subtree b contributes a factor at most $\rho^{|b|}$, with equality only for the five-cherry arm. So each vertex of a tree can contribute at most a factor ρ to π , and only vertices inside five-cherry arms achieve it. A maximizer therefore uses as many five-cherry arms as the arithmetic of n allows, and all the difficulty is in the lower-order terms. For comparison, Pant’s families grow like $(3/2)^{n/2} \approx 1.2247^n$, just below ρ .

1.3 How the proof goes

The proof has three layers.

- (a) *Algebra.* For a tree, the permanent of the Laplacian is a weighted count of matchings, and a cavity recursion computes it from the leaves up (Sections 2 and 3).
- (b) *Reduction to spiders.* Rooting a tree at a vertex of maximum degree, we show that any tree which is not a spider is strictly beaten by some spider with the same number of vertices (Section 5). The engine is the sharp inequality of Section 4, refined into per-vertex inequalities that measure how much each non-spider feature costs.
- (c) *Optimization over spiders.* Spiders form an explicit family with a closed-form value. We solve the optimization over it exactly (Section 6).

Each layer is split by size, because different arguments are efficient in different ranges (Figure 2):

n	reduction to spiders	best spider
4–6	exact rational evaluation of every tree	
7–149	envelope certificate (degree ≤ 23) + high-degree theorem	table sweep
150–491	credited rate cells + high-degree theorem	table sweep
≥ 492	rate cells + high-degree theorem	exchange lemmas + 223 polynomial certificates

1.4 How to read this paper

The mathematics is presented in the usual way, with proofs or proof outlines. Alongside it, shaded paragraphs marked **In Lean** say where each step lives in the formal development, what the formal

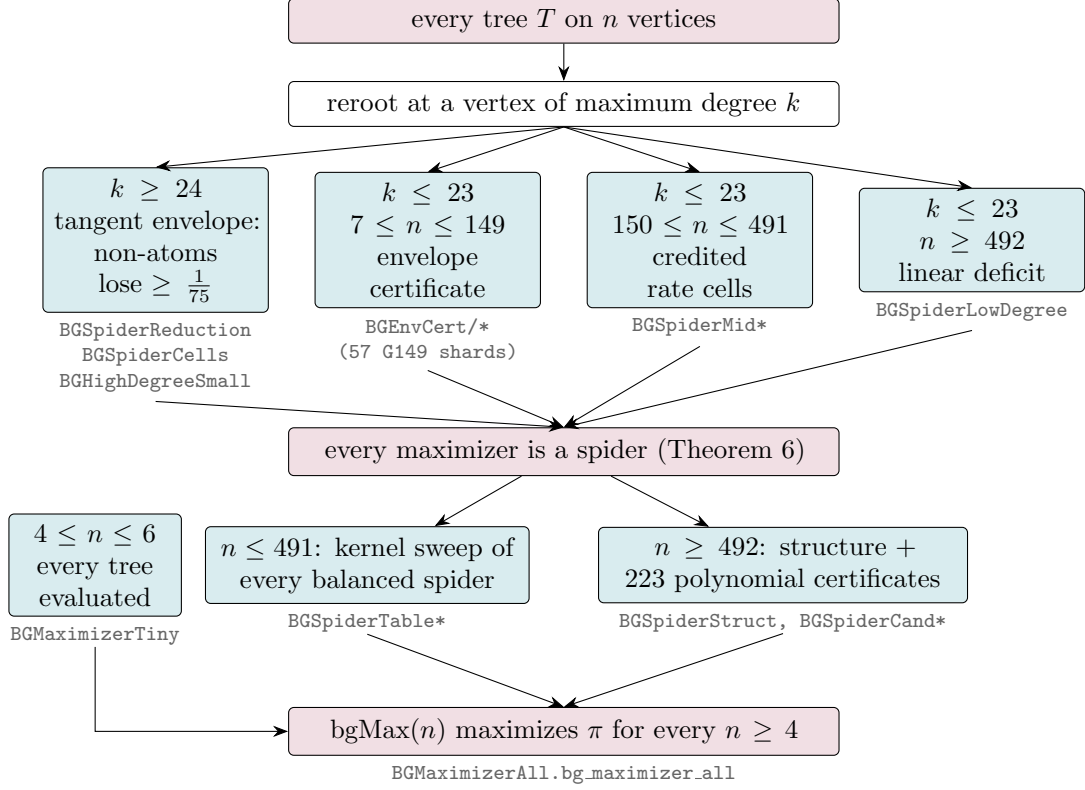


Figure 2: The proof at a glance, with the Lean modules that carry each step. The top half shows that every maximizer is a spider, the bottom half finds the best spider, and `BGMaximizerAll` joins the ranges.

statement is, and why it is organized the way it is. A reader interested only in the mathematics can skip them. A reader who wants to audit the formal proof should read them together with Section 7, which spells out exactly what the Lean theorem says, and with the reading guide `formalization/READING_GUIDE.md` in the repository.

Status. Every step is formalized in Lean 4 with Mathlib [7] and checked by the Lean kernel. The headline theorems depend only on `propext`, `Classical.choice` and `Quot.sound`, with no `sorry`, no `native_decide` and no added axioms. The result has not yet been refereed. The kernel certifies the deductions. What a reader must still check is that the formal statement says what we claim it says, and Section 7 is written to make that check easy. The work was carried out with extensive help from an AI system (see the acknowledgements). Partly for that reason we have tried to make every claim checkable by machine and every certificate reproducible from scratch.

2 From permanents to matchings

The first step is classical, and it is what makes the problem tractable at all: for a tree, the permanent of the Laplacian collapses to a sum over matchings.

Theorem 4 ([1, 3]). *For a tree T ,*

$$\text{per } L(T) = \sum_M \prod_{v \notin V(M)} \deg v, \quad \text{hence} \quad \pi(T) = \sum_M \prod_{uv \in M} \frac{1}{\deg u \deg v},$$

the sums running over all matchings M of T , including the empty one, and $V(M)$ denoting the set of matched vertices.

Proof. A term $\prod_i L_{i,\sigma(i)}$ of the permanent is nonzero only if every i is fixed by σ or sent to a neighbour. The non-trivial cycles of σ then run along edges of T . Since T has no cycles, they are transpositions along edges, that is, a matching M . A fixed point contributes $L_{ii} = \deg i$ and a matched edge contributes $L_{ij}L_{ji} = (-1)(-1) = 1$. Dividing by $\prod_v \deg v$ gives the second formula. $\square$

So $\pi(T)$ is a *weighted matching count*: every matching contributes the product, over its edges, of $1/(\deg u \deg v)$. The empty matching contributes 1; single edges contribute a little each; larger matchings contribute products of these small numbers (Figure 3). In statistical-physics language, π is the partition function of a monomer-dimer system in the sense of Heilmann and Lieb [4], with a dimer on edge uv carrying activity $1/(\deg u \deg v)$.

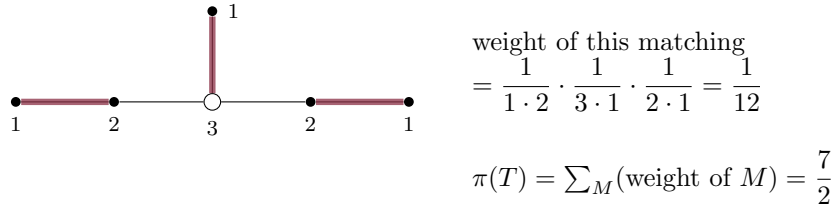


Figure 3: The spider C^2A_0 on six vertices (centre open, degrees shown by each vertex) and one of its matchings. Each matched edge uv contributes $1/(\deg u \deg v)$. Summing over all 12 matchings, including the empty one, gives $\pi(T) = \text{per } L(T) / \prod \deg v = 7/2$.

Two consequences are worth drawing out. First, $\text{per } L(T)$ is not hard to compute for trees: Theorem 4 and the recursion of the next section compute it in linear time. Every value in this paper is an exact rational that anyone can recompute. Second, the maximization problem now has a clear tension. A matched edge is worth more when its endpoints have small degree, but small degrees mean few edges, and every edge incident to a vertex raises that vertex’s degree and so lowers the weight of all its other edges. The maximizer resolves this tension in a surprisingly rigid way.

In Lean. This is `pi_eq_msum` in `Matching.lean`, proved for any finite acyclic simple graph with Mathlib’s `Matrix.permanent`. It is the only place where the permanent enters the formal proof. Everything afterwards works with the matching sum, called `Aobj`, and the bridge theorem `pi_utree` in `R47Tree.lean` says that for the trees of the formalization `Aobj t` equals $\text{per } L / \prod \deg$ of the realized graph. That is how the final theorem can be stated literally for the permanent (Section 7).

3 The cavity recursion

Matching sums on trees can be computed from the leaves upward. This is the “cavity method” of statistical physics, and it is the workhorse of the whole proof.

Planted branches. A *planted branch* b is a rooted tree whose root will be attached to one further vertex, its parent. Degrees are always measured in the final tree, so the root of b has degree $d_b = (\text{number of children}) + 1$. Let T_b be the matching sum of Theorem 4 restricted to edges of b ,

and T_b° the same sum over matchings that leave the root of b unmatched. The ratio

$$y_b = \frac{T_b^\circ}{d_b T_b}$$

is the *message* that b sends to its parent: it measures how available the root of b is to be matched upward, per unit of its degree. Splitting the matchings of b according to whether, and to which child, the root is matched gives the recursion

$$T_b = \left(\prod_c T_c \right) \frac{d_b + R_b}{d_b}, \quad y_b = \frac{1}{d_b + R_b}, \quad R_b = \sum_c y_c, \quad (1)$$

where the products and sums run over the children c of the root (Figure 4). Indeed $T_b^\circ = \prod_c T_c$, and matching the root to child c contributes $\frac{1}{d_b d_c} T_c^\circ \prod_{c' \neq c} T_{c'} = \frac{y_c}{d_b} \prod_{c'} T_{c'}$. A leaf has $T = 1$ and $y = 1$. For a tree rooted at a vertex with k children, the same computation with $d = k$ (the root has no parent) gives

$$\pi(T) = \left(\prod_c T_c \right) \left(1 + \frac{R}{k} \right), \quad R = \sum_c y_c. \quad (2)$$

Since (1) multiplies in one factor per vertex, a useful rewriting is

$$\log T_b = \sum_{v \in b} \log \left(1 + \frac{R_v}{d_v} \right). \quad (3)$$

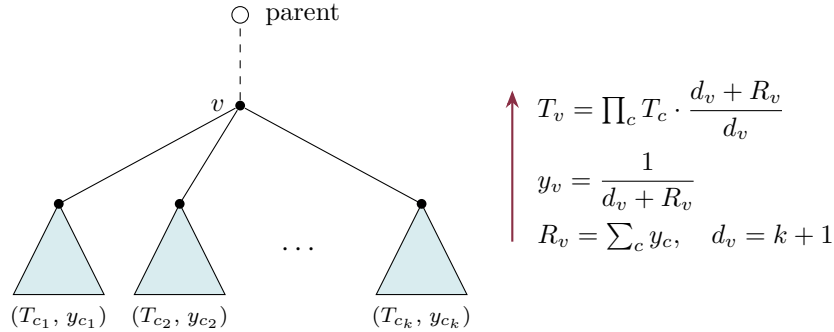


Figure 4: The cavity recursion. Each child subtree summarizes itself by a weight T and a message y , and the vertex combines them. The whole tree's ratio is the same combination at the root, with $d = k$ because the root has no parent.

Atoms. Iterating (1) on the smallest pieces:

piece	vertices	weight T	message y
leaf A_0	1	1	1
cherry	2	$3/2$	$1/3$
arm A_j	$2j + 1$	$(3/2)^j \frac{4j+3}{3(j+1)}$	$\frac{3}{4j+3}$

For the arm, the arm vertex has degree $j + 1$ and receives $R = j/3$ from its j cherries. In particular

$$T(A_5) = \left(\frac{3}{2} \right)^5 \cdot \frac{23}{18} = \frac{243}{32} \cdot \frac{23}{18} = \frac{621}{64},$$

and the arithmetic identity $27 \cdot 23 = 621$ hidden in this line will reappear as the reason the extremal structure is exactly tight.

Which piece gives the most weight per vertex? Figure 5 plots $\log T(A_j)/(2j+1)$. A cherry gives $\frac{1}{2} \log \frac{3}{2} \approx 0.2027$ per vertex. Arms do better, and the best is A_5 , at 0.2065862, with A_4 (0.2064721) and A_6 (0.2064696) just behind. As j grows the arm approaches the cherry value, because the arm vertex itself becomes a negligible fraction of the arm. This small plot already contains the answer’s shape: use A_5 ’s wherever possible, and fix the remainder with the nearly-as-good A_4 and A_6 . What the plot does *not* show, and what makes the problem hard, is that the root factor $1 + R/k$ in (2) also depends on the messages: arms with larger y (smaller arms) help the root. The rule in Theorem 2 is the outcome of that competition.

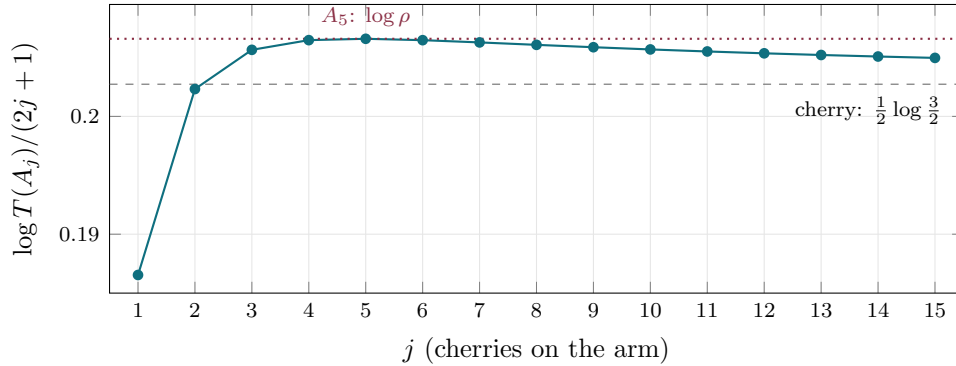


Figure 5: Weight per vertex of the arm A_j . The five-cherry arm is the unique maximum, and its value $\log \rho$ is the growth rate of the maximum of π (Theorem 3). The arms A_4 and A_6 trail it by about 10^{-4} , and long arms tend to the cherry value.

In Lean. The formalization has two models of trees and a bridge between them. `UTree` (file `R47Tree.lean`) is the “real” tree: a rooted tree that can be realized as a Mathlib `SimpleGraph`, which is where the permanent lives. `BGSCL.Branch` is a lightweight copy designed for recursion, and `BGSCL.cav` computes the pair (T_b°, T_b) by (1); `bY` is the message and `piRoot` is (2). The translation `fromU` from `UTree` to `Branch` and the theorem `Aobj_eq_piRoot` (file `BGMaximizer.lean`) say that computing with the lightweight model gives the true matching sum. Why two models? Proofs about permanents need a graph with a vertex type; proofs by recursion and by kernel computation need a plain inductive type with no graph attached. Keeping them separate, and proving once that they agree, keeps both kinds of proof simple.

4 The sharp ceiling and the growth rate

Write $F^* = \frac{1}{11} \log \frac{621}{64} = \log \rho$ and, for a planted branch b with $|b|$ vertices,

$$\ell(b) = \log T_b - |b| F^*.$$

Think of $\ell(b)$ as the *profit* of the branch: its log-weight minus the best possible log-weight for its size. The central inequality says profit is never positive.

Theorem 5 (sharp ceiling). *Every planted branch b satisfies $T_b \leq \rho^{|b|}$, that is $\ell(b) \leq 0$, with equality if and only if b is the five-cherry arm A_5 .*

4.1 Subactions: paying for a vertex with its children's credit

By (3), the profit is a sum of local terms,

$$\ell(b) = \sum_{v \in b} e_v, \quad e_v = \log \left(1 + \frac{R_v}{d_v} \right) - F^*.$$

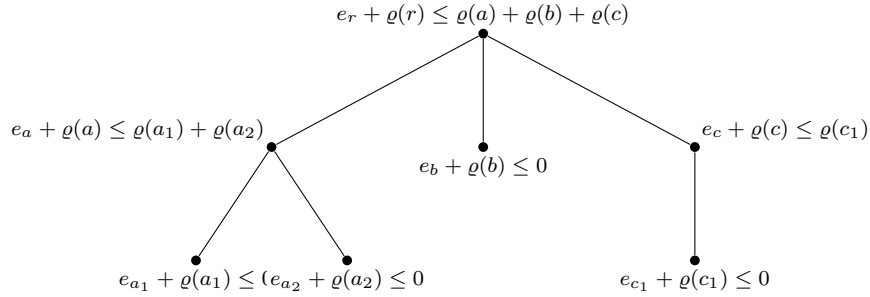
Some vertices have $e_v > 0$ (a vertex whose children send it large messages does well locally) and some have $e_v < 0$ (a leaf has $e = -F^*$). A term-by-term bound is hopeless. Instead we look for a *potential* $\varrho \geq 0$ on vertices, depending only on a vertex's local state, such that

$$e_v + \varrho(v) \leq \sum_{c \text{ child of } v} \varrho(c) \quad \text{for every vertex } v. \quad (4)$$

Read (4) as a budget: each vertex may run a local profit e_v , but must pay for it out of the credit $\varrho(c)$ banked by its children, and must also bank credit $\varrho(v)$ for its own parent. Summing (4) over all vertices of b , each $\varrho(v)$ appears once on the left and once on the right (at its parent), except the root's, so everything telescopes (Figure 6):

$$\ell(b) = \sum_v e_v \leq -\varrho(\text{root}) \leq 0.$$

In the language of ergodic optimization such a ϱ is a *subaction* [5]; its existence is the discrete analogue of finding a calibrating potential for an optimal orbit.



add them up: every ϱ except the root's cancels, so $\sum_v e_v \leq -\varrho(r) \leq 0$

Figure 6: Why a subaction proves the ceiling. Each vertex's inequality pays its local profit e_v out of its children's credit; summed over the tree, the credits cancel along every edge.

The witness. Remarkably, a very simple potential works. Writing y for a vertex's message and grouping vertices by their number of children,

$$\varrho_{\text{wit}} = \begin{cases} F^* & \text{leaf,} \\ 2F^* - \log \frac{3}{2} + \frac{1}{4}(y - \frac{1}{3}) & \text{one child,} \\ \frac{1}{32} y & \text{two children,} \\ \frac{1}{384} y & \text{three children,} \\ 0 & \text{four or more children.} \end{cases}$$

Three instances of (4) are equalities, and they are exactly the three kinds of vertex in A_5 :

- a leaf: $e = -F^*$ and $\varrho = F^*$, so $e + \varrho = 0$;
- the middle vertex of a cherry (one leaf child, message $1/3$): $e + \varrho = (\log \frac{3}{2} - F^*) + (2F^* - \log \frac{3}{2}) = F^* = \varrho(\text{leaf})$;
- the arm vertex with five cherries ($d = 6$, $R = 5/3$, $\varrho = 0$): $e = \log \frac{23}{18} - F^*$, and the children's credit is $5(2F^* - \log \frac{3}{2})$. These agree exactly when $11F^* = \log(\frac{23}{18}(\frac{3}{2})^5) = \log \frac{621}{64}$, which is the definition of F^* .

Every other instance of (4) holds with room to spare. Checking that is a finite computation once one notices that (4) only involves a vertex and its children's local states, and that large degrees are easy (the term $\log(1 + R/d)$ is bounded, while the credit of the children grows with their number). The verification splits into cells by degree and by ranges of the children's messages; in each cell the inequality compares a rational envelope against a rational enclosure of a logarithm.

Equality. If $\ell(b) = 0$ then every inequality along b is tight. Tightness at the root forces degree six, and at degree six it forces all five children to be cherries: a single non-cherry child makes the cell strict. The strictness needs the numerical fact $2F^* - \log \frac{3}{2} < \frac{1}{96}$, which follows from $\log x < x - 1$ at $x = (621/64)^2(2/3)^{11} \neq 1$. At degree three the non-strict version of this bound is exactly tight, so it cannot be weakened.

In Lean. The subaction bridge (`ceiling_of_witness`) and `pwit` are in `BGSCLSubaction.lean`; the cells are in `BGSCLSubactionD4Cells.lean` (35 cells for vertices with four children), `BGSCLSubactionTailWrap.lean` (high degree) and the dispatcher `BGSCLSubactionDispatch.lean`, which sorts a vertex's children and sends it to the right cell. The logarithm enclosures are Taylor or `atanh` series with explicit remainders, proved as ordinary Mathlib lemmas, so no floating point is trusted. `BGSCLSharp.bg_sharp` packages both halves of Theorem 5.

4.2 The growth rate

Proof of Theorem 3. Root T anywhere. In (2) each child c is a planted branch, so $\prod_c T_c \leq \rho^{n-1}$ by Theorem 5; and each $y_c \leq 1$, so $1 + R/k \leq 2$. For the lower bound, take a centre carrying $K = \lfloor (n-1)/11 \rfloor$ arms A_5 , and fill the remaining at most ten vertices with cherries and at most one leaf; each has $T \geq 1$, so $\pi \geq \rho^{11K} \geq \frac{64}{621}\rho^n$. $\square$

In Lean. `R3Cert.Step3.bg_max_growth` in `BGGrowthRate.lean`. It is stated for `UTree` and `Aobj`, so via `pi_utree` it is a statement about permanents.

Figure 7 shows $\pi(\text{bgMax}(n))/\rho^{n-1}$ for $n \leq 1500$. After a transient it settles into a band near 1.13, with a period-11 wiggle that reflects the residue rule. The two bounds of Theorem 3 are far apart in constant but equal in rate; the whole remaining effort of the paper is to determine the constant, and the tree, exactly.

5 Every maximizer is a spider

We now show that a tree that is not a spider cannot be a maximizer. The idea is to measure precisely how much each departure from spider shape costs, and to show the cost always exceeds any possible gain.

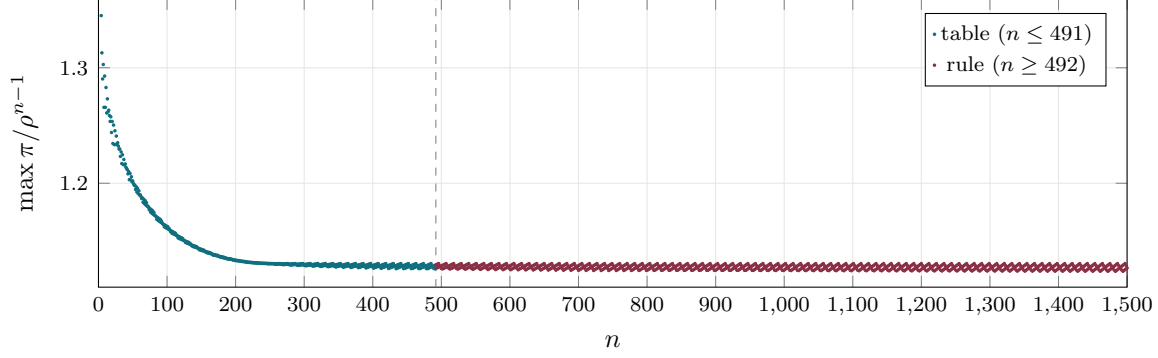


Figure 7: The normalized maximum $\pi(\text{bgMax}(n))/\rho^{n-1}$. Theorem 3 confines it to $[\frac{64}{621}\rho, 2]$; the true values lie between 1.12 and 1.35 and oscillate with period 11 for large n .

5.1 Setting up the comparison

Rooting at maximum degree. Rerooting a tree changes neither π nor the vertex count, so we may root at a vertex of maximum degree k . Then every vertex has at most $k - 1$ children. This cap is what makes finitely many local inequalities sufficient for each k . Call the root's children $b_1, \dots, b_k$. The *atoms* are the leaf, the cherry and the arms A_j ; the tree is a spider centred at the root exactly when every b_i is an atom (Figure 8).

The profit identity. Taking logarithms in (2),

$$\log \pi - (n - 1)F^* = \sum_{i=1}^k \ell(b_i) + \log \left(1 + \frac{S}{k}\right), \quad S = \sum_i y_{b_i}. \quad (5)$$

By Theorem 5 every term of the sum is ≤ 0 , so a tree's total profit is its root bonus $\log(1 + S/k)$ minus the losses of its branches. A non-atom branch has a strictly negative profit, but it might send a larger message and so raise the root bonus. We must show that trade never pays.

The benchmark. We compare against explicit spiders. The A_5/A_4 spider, with $q \equiv 5(n - 1) \pmod{11}$ arms A_4 and the rest A_5 , satisfies

$$\log \pi - (n - 1)F^* \geq \log \frac{26}{23} - \frac{10}{960}, \quad (6)$$

because each A_4 costs little profit and sends a larger message than an A_5 ($y(A_4) = \frac{3}{19} > \frac{3}{23} = y(A_5)$).

The argument then splits by the root degree k and by n .

5.2 High degree: $k \geq 24$

When the root has many children, the root bonus $\log(1 + S/k)$ is nearly linear in S over the relevant range. Replace it by its tangent line at the spider value $S/k = 3/23$ (Figure 9). The slope prices each unit of message at $\mu = \frac{23}{26k} \leq \frac{23}{624}$, and the profit identity becomes a sum over branches of

$$V_\mu(b) = \ell(b) + \mu y_b,$$

a branch's profit plus the market value of its message. On this range of prices we show that A_5 maximizes V_μ over all branches, and that every non-atom lies below it by at least $\frac{1}{75}$. (The closest

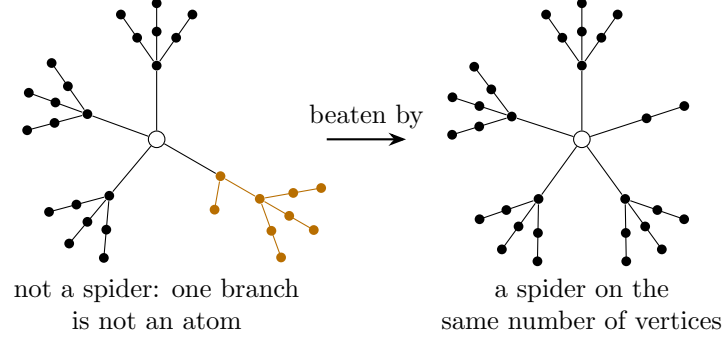


Figure 8: The reduction. Rooted at a vertex of maximum degree, a tree is a spider exactly when every branch at the root is an atom (leaf, cherry or arm). A non-atom branch (highlighted) always costs more profit than it can return through the root bonus, so some spider of the same size does strictly better.

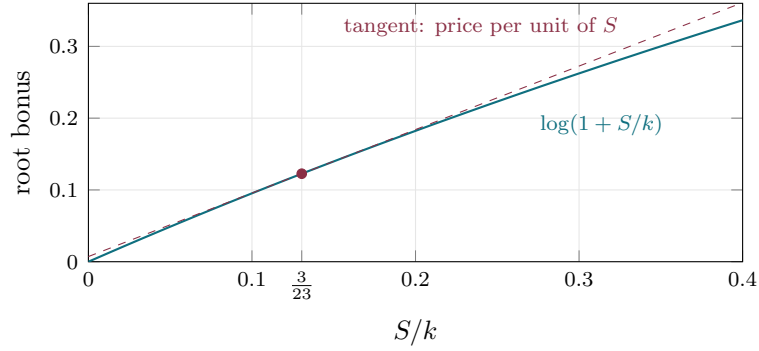


Figure 9: The high-degree argument replaces the concave root bonus by its tangent at the spider value $S/k = 3/23$, which can only overestimate it. The bonus then becomes linear, and each branch can be judged on its own: its profit plus the value of its message at a fixed price.

non-atom competitor, at 0.01452 below, is a vertex carrying five cherries and one A_4 .) So a tree with a non-atom branch has $\log \pi - (n-1)F^* \leq \log \frac{26}{23} - \frac{1}{75}$, which is below the benchmark (6) because $\frac{1}{75} > \frac{10}{960}$.

In Lean. The envelope statement reduces, through the subaction of Section 4, to three per-vertex inequalities (`surchargerCore.proved`, `atomCell0Core.proved`, `atomCellMuCore.proved`) in `BGSpiderCells.lean`, proved with 22 rational tangent certificates and degree-six Taylor enclosures of $\log$; the tightest margin is $7.3 \cdot 10^{-4}$. The result is `spider_dominates_highDegree_uncond` for $n \geq 91$. For $25 \leq n \leq 90$ the benchmark (6) is too weak, so `BGHighDegreeSmall.highDegree_small` compares instead with the exact best spider from the table of Section 6, one kernel-checked inequality per n .

5.3 Low degree and large n : linear deficits

When $k \leq 23$ the tangent trick is too crude, and we use size instead. A tree with a non-atom branch should lose profit in proportion to its size, while the root bonus is bounded. We strengthen the subaction inequality (4) by a small fixed rate α per vertex. A *rate cell* with degree cap D and rate α is a per-vertex inequality which, telescoped exactly as in Figure 6, yields a *linear deficit*

$$\ell(b) + \varrho_{\text{wit}}(b) \leq -\alpha|b| \quad \text{for every non-atom branch } b \text{ with at most } D-1 \text{ children per vertex.}$$

Atoms are exempt, because A_5 has zero profit and cannot lose anything. Then

$$\log \pi - (n-1)F^* \leq R_k - \alpha(n-1),$$

where R_k bounds what the root can gain. With $R_k \leq 0.319$ and the rates below, this falls under the benchmark once $n-1 \geq 491$. That is where the threshold 492 comes from.

root degree k	6–23	5	4	3	2
cap D	23	5	4	3	2
rate α	1/3700	1/2100	1/660	1/420	1/135

In Lean. `BGSpiderLowDegree.lean`: 32 rate cells and 22 root cells, each a tangent certificate with rational envelopes (tightest margin $2.8 \cdot 10^{-4}$), assembled into `spider_dominates_lowDegree_uncond`. `BGMax.bg_spider_reduction` in `BGMaximizer.lean` combines both degree ranges with the rerooting lemma `exists_maxRooted` for every $n \geq 492$.

5.4 Low degree, $150 \leq n \leq 491$: rate cells with credits

Below 492 the linear deficit is not yet large enough, and we sharpen it in two ways. First, in a tree of maximum degree $\leq D$ every message satisfies $y \geq 1/(2D-1)$, which narrows the range of messages each cell must handle. Second, non-atom branches carry a *credit* κ depending on their number of children, $\ell(b) + \varrho_{\text{wit}}(b) \leq -\alpha|b| - \kappa(b)$, which they pass up to the root. The root is then bounded by an explicit knapsack over atoms, certified separately for each pair (k, n) .

In Lean. `BGSpiderMid.lean` proves the generic credited telescoping. The generated files `BGSpiderMidCellsD2–D23` (cells, one file per cap) and `BGSpiderMidRootK2–K23` (root knapsacks, one file per root degree) are the certificates. `BGSpiderMidFinal.lean` assembles `spider_dominates_of_maxDegreeRoot_150`, and `BGMax.maximizer_is_spider_150` states the result for every $n \geq 150$.

5.5 Low degree, $7 \leq n \leq 149$: the envelope certificate

For small trees a uniform rate is far too weak, so we compute instead. The object computed is a *Bellman envelope*: a table $w(m, g)$ that bounds, for every branch of size m , its profit plus the value of its message at price μ_g , where the prices run over a grid (Figure 10).

Fix a rational rate f close to F^* (it cancels between a tree and a spider of the same size), put $V_\mu(b) = \log T_b - |b|f + \mu y_b$, and consider a vertex with its children. The function $x \mapsto \log x + \mu/x$ is concave on the relevant range, so its tangent bounds V_μ at the vertex by a constant plus $\sum_c V_\nu(c)$ at a new price ν . In words: a vertex's value at price μ is controlled by its children's values at a price that the vertex determines. The children's sizes add up to $m - 1$, so the best total over children is a max-plus knapsack over sizes. A table that satisfies all these knapsack inequalities is a valid envelope by induction on size, and the proof needs only that finitely many table inequalities hold.

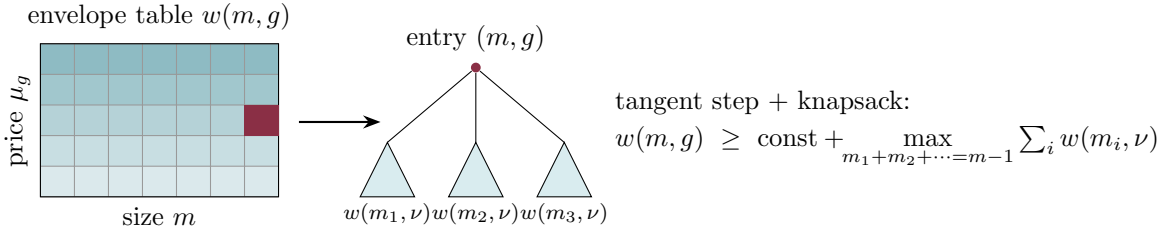


Figure 10: The envelope certificate. A table indexed by branch size and price is valid if every entry dominates the tangent-step bound computed from smaller entries; induction on size then bounds every branch. The table for $n \leq 149$ is stored in fixed point and checked entry by entry by the Lean kernel.

We cover root degrees $2 \leq k \leq 23$, using separate tables for vertex-degree caps $C \in \{1, \dots, 8, 12, 16, 22\}$; roots of degree ≥ 24 are handled by the high-degree argument. The final root inequality, with at least one non-atom child, is compared against certified lower bounds for explicit spiders.

In Lean. This is the most computation-heavy part of the proof, and its organization is dictated by how the Lean kernel evaluates. The theory is in `BGEnvCert/`: **Tangent** (the tangent step), **Envelope** (the induction on size), **Sound** (a passing Boolean check implies the real inequalities), **Spider** (certified spider values) and **AssembleK** (combining the caps). The data is in `BGEnvCert/G149/`: 57 generated files. Tables are fixed point with 34 fractional bits, packed as lanes of large natural numbers, and each check is a Boolean function evaluated by `decide +kernel`. The file `MainK` assembles them into `envcertK`, and `BGMaximizerSmall.maximizer_is_spider_small` gives the result for $7 \leq n \leq 149$. Section 8 explains why the data is split the way it is.

5.6 The smallest trees, and the summary

For $4 \leq n \leq 6$ there are few enough trees to evaluate every one exactly over $\mathbb{Q}$.

Theorem 6. *For every $n \geq 7$, every tree on n vertices that maximizes π is, after rerooting at a vertex of maximum degree, a spider. For $n \geq 492$ every tree on n vertices is weakly beaten by a spider on n vertices.*

In Lean. `BGMaximizerTiny.lean` enumerates trees for $n \leq 6$ by a fuel-bounded structural recursion (the kernel does not efficiently unfold well-founded recursion) and evaluates them with a $\mathbb{Q}$ copy of the cavity recursion, `cavQ`, proved equal to the real one by `cavQ_cast`. The three range theorems of Theorem 6 are `maximizer_is_spider_small`, `maximizer_is_spider_150` and `bg_spider_reduction`. Existence of a maximizer at each size, which the argument needs, is `BGMaximizerMid.exists_max_tree`: there are finitely many trees on n vertices.

6 The best spider

We now optimize over spiders. By (2) and the table of atoms, a spider's value has a closed form, and the problem becomes a question about integers.

6.1 The closed form and the balance identity

A spider is described up to isomorphism by the multiset of its centre's children, and

$$F = \left(\frac{3}{2}\right)^{c+\sum j} \prod_{\text{arms}} \alpha_j \cdot \left(1 + \frac{c/3 + \sum b_j}{D}\right), \quad \alpha_j = \frac{4j+3}{3(j+1)}, \quad b_j = \frac{3}{4j+3},$$

where c is the number of cherries at the centre, the product and sum run over the arms (leaves being A_0 , with $\alpha_0 = b_0 = 1$), and D is the degree of the centre. The first factor is fixed by n up to the number of arms, so the optimization is a balance between the arm factors α_j and the bracket.

Lemma 7 (balance). *Put $Q_c(x, y) = \alpha_x \alpha_y (c + b_x + b_y)$. For integers $k, t \geq 0$,*

$$Q_c(k+t+1, k+1) - Q_c(k+t+2, k) = \frac{(t+1)((c-3)(8k+4t+15)+3)}{9(k+1)(k+2)(k+t+2)(k+t+3)}.$$

Consequently, if $D \geq 3$, moving a cherry from an arm to an arm at least two smaller strictly increases F , and in every maximizer with $D \geq 3$ all arm sizes (leaves included) differ by at most one.

Proof. The move (Figure 11) keeps n , D and the power of $\frac{3}{2}$, and multiplies F by $Q_c(\text{new})/Q_c(\text{old})$ with $c = D + R_0 \geq 3$, where R_0 is the message sum of the other children. The right side of the identity is then positive. The identity itself is a polynomial identity in k, t, c . $\square$

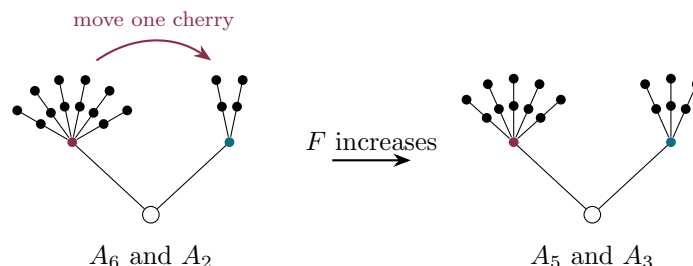


Figure 11: The balance move of Theorem 7, shown with $k = 2, t = 2$ (the centre's other children are not drawn). Whenever the centre has at least three children, evening out two arms whose sizes differ by two or more strictly increases the ratio. So a best spider has all its arms of two consecutive sizes.

Two further exact exchanges are used: two leaves at the centre are never better than one cherry, and a general local-exchange criterion that is affine in R_0 , and so reduces to checking two quadratics in the number D_0 of other children.

In Lean. `BGSpiderOpt.lean` models a spider as a list of children with the closed-form value F over $\mathbb{Q}$ and proves `balance_identity`, `F_balance_lt`, `balanced_of_isMax`, `F_leafpair_lt` and `F_exchange_lt`. The link to real trees is `BGSpiderRule.Aobj_spiderU`: for the tree `spiderU 1` built from a child list `1`, `Aobj` equals $F 1$.

6.2 Sizes up to 491: a kernel sweep

By Theorem 7, a maximizer with $D \geq 3$ is determined by four numbers (c, s, a, b) : c cherries, a arms A_s and b arms A_{s+1} . For each n there are only a few thousand such configurations, plus the configurations with $D \leq 2$. Each value has an explicit integer numerator and denominator. For each $n \leq 491$ the kernel compares every candidate against the table entry by a cross-multiplied inequality of natural numbers.

At $n = 21$ two non-isomorphic spiders tie (C^{10} and $C^3 A_3^2$); the table records $C^3 A_3^2$. The table also shows how varied small maximizers are. Below $n = 46$ arms of every size from one to eight appear, with up to eight cherries at the centre. From $n = 46$ on every arm is an A_4 , A_5 or A_6 , but a few cherries remain at the centre until $n = 333$, and the A_4/A_6 choice keeps shifting until $n = 2319$.

In Lean. `BGSpiderTable.lean` proves the soundness of the check (`spider_opt_of_check`); the table itself is `BGSpiderTableData.tabData`, and the 89 generated files `BGSpiderTableChunk.*` run the check for about five sizes each, assembled by `BGSpiderTableAll.spider_opt.table`. The table rows are printed in Section A, generated from the Lean file itself.

6.3 Sizes from 492: structure, then 223 certificates

For large n a sweep is impossible, so we argue. Let l be a spider maximizing F with $n \geq 492$. A sequence of exchanges pins down its shape:

1. the centre has at least three children (otherwise a global bound shows it is far from optimal);
2. no arm A_j has $j \geq 13$: split it into A_{j-11} and two A_5 's;
3. no arm has $7 \leq j \leq 12$: trade eleven A_j 's for $2j + 1$ copies of A_5 , and compare globally with the rule spider;
4. at most eight cherries: nine cherries lose to two A_4 's;
5. no arm has $j \leq 3$;
6. hence every arm is A_4 , A_5 or A_6 ; by balance A_4 and A_6 never occur together; and there are at most ten of each.

The global comparison uses the identity $(\prod T)^2 = (3/2)^{n-1} \prod h$ with $h(\text{cherry}) = 1$ and $h(A_j) = \frac{2}{3}\alpha_j^2$; its tightest instance is $24.45 < 28.43$.

What remains is a finite list of shapes whose sizes grow with n . A candidate is determined by its counts (c, k_4, k_5, k_6) , and

$$F = \Phi(c, k_4, k_5, k_6) = \left(\frac{3}{2}\right)^c G_4^{k_4} G_5^{k_5} G_6^{k_6} \left(1 + \frac{c/3 + k_4 B_4 + k_5 B_5 + k_6 B_6}{c + k_4 + k_5 + k_6}\right)$$

with $G_4 = \frac{513}{80}$, $G_5 = \frac{621}{64}$, $G_6 = \frac{6561}{448}$, $B_4 = \frac{3}{19}$, $B_5 = \frac{3}{23}$, $B_6 = \frac{1}{9}$. The triple (c, k_4, k_6) fixes the residue class s , hence the rule winner's counts. After dividing out a common power of G_5 , the comparison $\Phi(\text{candidate}) \leq \Phi(\text{winner})$ becomes $P(m) \geq 0$ for an explicit quadratic P in the common number m of five-cherry arms. There are 189 triples and 223 certificates. 221 of them write $P(L+t)$ with nonnegative coefficients in t , which is visibly positive for every $t \geq 0$. The other two, the competitions that decide the thresholds 722 and 2319, write it in the Bernstein basis on a bounded interval, because there the rival really does win outside the interval (Figure 12).

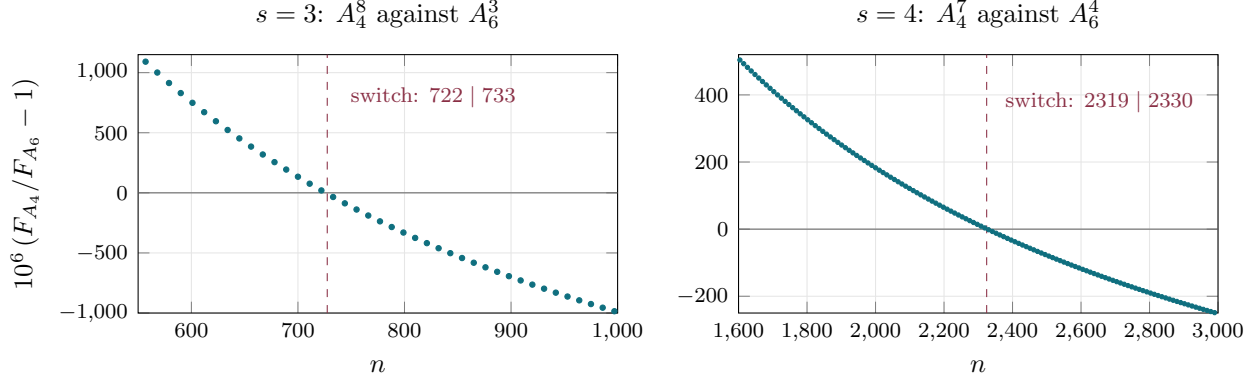


Figure 12: The two late switches in the rule. In residue classes $s = 3$ and $s = 4$ the maximizer uses A_4 's until the relative advantage of A_6 's, which grows slowly with n , overtakes it. The margins are tiny: at the last size before each switch the A_4 spider wins by $2.0 \cdot 10^{-5}$ and $3.6 \cdot 10^{-6}$ in relative terms. The two Bernstein-basis certificates prove these signs exactly on each side.

In Lean. `BGSpiderStruct.structProp_492` proves the shape constraints (the local exchanges close by `nlinarith` once certain quadratics are shown positive). `BGSpiderCandBase.lean` defines Φ and P . The generated files `BGSpiderCandCells0–8` hold the certificates, one file per cherry count, each checked by `ring` (the identity) and `norm_num` (the signs), and `BGSpiderCand.candProp_492` assembles the 189 cases. `BGSpiderRule.bg_maximizer_of` combines these with the reduction, and `BGMaximizerFinal.bg_maximizer` is the theorem for $n \geq 492$.

Together, Theorem 6 and this section prove Theorem 2.

7 What exactly the Lean theorem says

The main theorem in Lean is

```
theorem R3Cert.BGMaximizerAll.bg_maximizer_all (n : Nat) (h4 : 4 <= n) :
  use (bgMax n) = n /\ forall t : UTree,
    use t = n -> Aobj t <= Aobj (bgMax n)
```

and its companion `bg_maximizer_all_perm` states the same inequality for the permanent of the Laplacian divided by the product of degrees, computed on the realized graphs. The proof of `bg_maximizer_all` is a ten-line case split on n that calls the four range theorems. What a reader must trust is not that proof (the kernel checked it) but the definitions it is about:

- `UTree` is the type of finite rooted trees, built by `node` from a list of subtrees, and `use` counts vertices. Every finite tree, rooted anywhere, is a `UTree`; rerooting is handled inside the proof, so the quantifier really ranges over all trees.

- `Step3.realize (dtRealize t)` turns t into an explicit edge list, and `aGraph` turns that into a Mathlib `SimpleGraph` on its vertex type.
- `lapl G` is the matrix with $\deg i$ on the diagonal, -1 on edges and 0 elsewhere, and `Matrix.permanent` is Mathlib’s permanent.
- `Aobj t` is the weighted matching sum of the realized tree, and `pi.utree` proves that it equals $(\text{lapl } (\text{aGraph } \dots)).\text{permanent}$ divided by the product of the degrees.
- `bgMax n` is `spiderU (tab n)` for $n \leq 491$ and `spiderU (W n)` otherwise, where `tab` reads the table `BGSpiderTableData.tabData` and `W` implements the rule of Theorem 2.

So `bg_maximizer_all_perm` states: for every $n \geq 4$ and every tree t on n vertices, per $L / \prod \deg$ of t is at most that of `bgMax(n)`, with the permanent and the degrees read off the actual graph. The file `AxiomGuard.lean` prints the axioms of the 17 headline theorems. A `sorry` anywhere in their dependencies would appear there as `sorryAx`, and a `native_decide` as `Lean.ofReduceBool`.

What is not claimed in Lean. The Lean theorem gives *a* maximizer. That it is unique up to isomorphism (except at $n = 21$) is established by exact computation for $n \leq 491$ (Section 9) and for $n \geq 2320$ by an exact rational certificate, but uniqueness is not formalized.

8 Certificates and Telperion

Most of the proof’s bulk is data: envelope tables, rate-cell constants, table rows, polynomial coefficients. We produce it with generators written in Python using exact integer and rational arithmetic, and we never trust the generators. Each certificate is a Lean file whose checks the kernel evaluates, so a wrong certificate is simply a failed build.

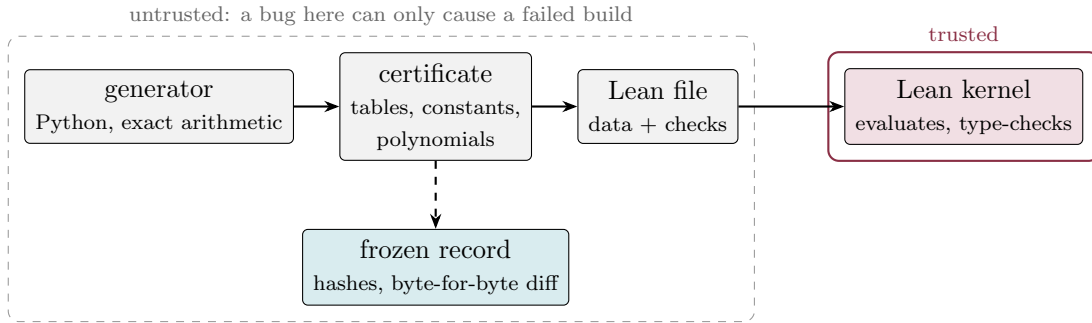


Figure 13: The certificate pipeline. Only the kernel is trusted. The frozen record lets anyone regenerate a family from scratch and confirm that the Lean files in the repository are exactly what the generator produces.

The certificates are packaged with Telperion, a pipeline built on exactly that division of labour: an untrusted *emitter* produces a certificate, a small adapter renders it as Lean, and the Lean kernel is the only trusted component (Figure 13). For each certificate family the repository contains the generator, a *frozen* copy of its output with hashes, and a driver that regenerates the family from scratch and compares it byte for byte, both with the frozen copy and with the files in the formalization:

family	Lean files	content
<code>spider_table</code>	91	table rows and the per- n sweep, $n \leq 491$
<code>envcert_g149</code>	57	envelope tables, tangents, spider bounds, $7 \leq n \leq 149$
<code>mid_cells</code>	45	credited rate cells and root knapsacks, $150 \leq n \leq 491$
<code>cand.certs</code>	10	the 223 polynomial certificates, $n \geq 492$

The byte-for-byte check matters for review. A reader does not have to read 57 files of packed integers; it is enough to read the generator, run it, and see that it reproduces the files the kernel checked. And even that is only needed to understand the proof, not to trust it: the kernel’s verdict does not depend on how the data was produced.

Making the kernel do it. Three facts about the Lean kernel shaped the certificates.

- (i) The kernel unfolds definitions by well-founded recursion very slowly, so every recursive check is structural, with an explicit fuel parameter.
- (ii) It does not share the evaluation of repeated argument subterms: a loop state used twice is evaluated twice, and a chain of such loops blows up. We emit literal constants, and force evaluation of natural numbers before passing them on (`BGEnvCert.Force`).
- (iii) Arithmetic in $\mathbb{Q}$ is slow in the kernel, so large checks use natural numbers and fixed point, and only small ones use $\mathbb{Q}$.

We use `decide +kernel` and never `native_decide`, so the compiler is not trusted. Because the kernel keeps every intermediate term of a declaration in memory, large checks are split into many small theorems and many files, and a unifying file assembles them. Even so, the heaviest files need 15–35 GiB each. The sharding is what makes the proof checkable on a single large workstation.

9 Independent computations

None of the following is needed for the Lean proof. They are independent checks of the answer by entirely different methods.

- *Certified search*, $n \leq 491$. A bottom-up search over all rooted trees, with no degree cap, storing every quantity as an outward-rounded interval and discarding a partial state only when it is provably strictly beaten in every completion. Surviving candidates are re-evaluated in exact rational arithmetic. For every $n \leq 491$ the exact maximum equals the table value, and *every* maximizing tree is a spider. (952 s on one core.)
- *Brute force*. Exact enumeration of all non-isomorphic trees agrees for $n \leq 18$.
- *Spider optimization*. An exhaustive exact optimization over the spider family for $n \leq 2400$, together with an exact rational certificate for $n \geq 2320$, agrees with the table and the rule; an exact Pareto dynamic programme over the family with no balance assumption agrees for $n \leq 600$.

10 Discussion

10.1 What the answer says

The maximizer is *rigid*. For large n it is a star of identical eleven-vertex blocks with a bounded number of defects, at most ten, whose type is dictated by $n \bmod 11$. The value per vertex is set by a single local structure, the five-cherry arm, and the global optimum is as close to “all five-cherry arms” as divisibility allows. This is the typical shape of answers in ergodic optimization, where an optimal orbit is periodic and a subaction certifies it, and indeed that is how the proof of the growth rate works: the potential ϱ_{wit} is tight exactly on the three vertex types of the optimal block. It would be interesting to know how general this is. For which other weighted matching problems on trees, or other tree functionals computed by a cavity recursion, is the extremal tree a periodic arrangement of a single block, certified by a finite piecewise-affine subaction?

The answer also explains why the problem resisted guesses. The earlier conjectures were natural extrapolations from small trees, and the table in Section A shows why that is treacherous. For small n the maximizer wanders through arms of many sizes with many cherries at the centre. It settles into A_4 ’s, A_5 ’s and A_6 ’s only at $n = 46$, keeps a few centre cherries until $n = 333$, and the A_4/A_6 competition keeps changing until $n = 2319$. A conjecture based on data up to $n = 100$, or even $n = 1000$, would have been wrong in its details.

It is also worth noting what the maximizer is *not*. The earlier candidates, subdivided stars and Pant’s caterpillars, spread their high-degree vertices along a path. The maximizer does the opposite: it has a single hub, and every other vertex is within distance three of it. In the language of the cavity recursion, the hub is where the messages of many arms are pooled, and pooling pays because $\log(1 + R/k)$ rewards a large total message at one vertex.

10.2 Permanents

The permanent is #P-hard, and its extremal theory is thin: questions of the form “which matrix in a natural class maximizes the permanent?” are often hard even when the class is simple. For Laplacians of trees, the matching-sum formula turns the permanent into a partition function that factorizes over the tree, and that is what made this problem solvable. The same structure applies to other matrix functions of trees built from D and A . For example, the signless Laplacian $D + A$ of a tree has the same permanent, since a matched edge contributes $A_{ij}A_{ji} = 1$ either way, so Theorem 2 answers the analogous question for it too. Immanants of L interpolate between the determinant and the permanent, and orderings of trees by such quantities have been studied [6]. The Brualdi–Goldwasser question for bipartite graphs, where cycles appear and the matching-sum formula fails, and for trees with a prescribed degree sequence, are natural next targets.

10.3 Method

Most of the proof is a small number of ideas (the cavity recursion, the subaction, tangent envelopes, exchange arguments) applied through a large number of verified inequalities. This is a common shape for extremal results, and such proofs are often hard to check: the ideas are in the paper, but the inequalities are in a computer program that a referee cannot easily audit. Here every inequality is checked by the Lean kernel, and every certificate can be regenerated and compared byte for byte. We think this division, conceptual proof for people and certificate checking for the kernel, is a good model for computer-assisted extremal combinatorics, and the practical lessons of Section 8 (sharding, fixed point, structural recursion) apply well beyond this problem.

A related point concerns how the work was done. Much of the exploration, formalization and certificate engineering was carried out with an AI system. In a project of that kind the natural worry is an error that no one notices. Formal verification changes that: an error in a certificate or a proof step is a failed build, not a hidden mistake. The remaining human responsibility, checking that the definitions and the statement mean what the paper says, is exactly the part that Section 7 isolates.

10.4 What remains

- *A conceptual proof of the reduction.* The thresholds 150, 491 and 492 are artefacts of the method. A direct argument that every maximizer is a spider, for all n , would be more illuminating than the certificates, and would probably say something about when cavity-recursion optimizers are “star-shaped”.
- *Uniqueness in Lean.* Uniqueness (except at $n = 21$) is known by computation and certificate but not formalized.
- *The constant.* Figure 7 suggests that $\max \pi/\rho^{n-1}$ approaches a periodic pattern of period 11. Since the maximizer is explicit this is a computation in principle, but a clean closed form for the limit points would be nice.
- *Independent review.* The kernel checks the deductions; human review of the statement and the definitions is still needed, and welcome.

11 Limitations

- The result has not been refereed. The Lean kernel checks the deductions; the correspondence between the formal definitions and the mathematical statement (Section 7) is what a human reviewer should check.
- The proof is long and mostly computational. We do not have a short conceptual proof that the maximizer is a spider, only per-vertex inequalities verified by certificate.
- Uniqueness is not formalized.
- The trusted base is the Lean kernel, Mathlib’s definitions of the permanent and of simple graphs, and our definitions listed in Section 7.
- The full build needs a machine with at least 64 GiB of memory and several CPU-hours.

12 Reproducing the results

The repository <https://github.com/DrMurphyIsIn/brualdi-goldwasser> contains the Lean project (Lean v4.32.0, Mathlib), the certificate generators with their frozen outputs, the Telperion engine, the source of this paper and a project page with an interactive version of Theorem 2 (<https://drmurphyisin.github.io/brualdi-goldwasser/>).

```
cd formalization && lake exe cache get && lake build -j 4
lake env lean AxiomGuard.lean
certificates/verify.sh          # regenerate all certificates, compare
certificates/verify.sh --full  # also the certified search, n <= 491
```

The plots in this paper are generated from the same data by `paper/figdata.py`, and the appendix table by `paper/gen_table.py`.

Acknowledgements

The formalization, the certificate generators and much of the exploration were developed with extensive assistance from Claude, an AI system made by Anthropic, working under the author's direction. The author takes responsibility for the content. The correctness of the formal proof rests on the Lean kernel, not on either.

Licensing

Mathematics and Lean code: Apache-2.0. This paper: CC-BY-4.0. The vendored Telperion engine: Business Source License 1.1, free for research, teaching and peer review.

References

- [1] R. A. Brualdi and J. L. Goldwasser, *Permanent of the Laplacian matrix of trees and bipartite graphs*, Discrete Math. **48** (1984), 1–21.
- [2] T. Wu, X. Dong and H.-J. Lai, *Two problems on Laplacian ratios of trees*, Discrete Appl. Math. **372** (2025), 224–236.
- [3] P. Pant, *Counterexamples to a conjecture on Laplacian ratios of trees*, arXiv:2605.14176 (2026).
- [4] O. J. Heilmann and E. H. Lieb, *Theory of monomer-dimer systems*, Comm. Math. Phys. **25** (1972), 190–232.
- [5] T. Bousch, *La condition de Walters*, Ann. Sci. Éc. Norm. Supér. **34** (2001), 287–311.
- [6] P. Csikvári, *On a poset of trees*, Combinatorica **30** (2010), 125–137.
- [7] The mathlib Community, *The Lean mathematical library*, CPP 2020, 367–381.

A The maximizers for $4 \leq n \leq 491$

Notation: C^c is c cherries at the centre, A_j^m is m arms with j cherries, L a leaf at the centre. The third column is π/ρ^{n-1} , which by Theorem 3 lies in $[\frac{64}{621}\rho, 2]$. At $n = 21$, C^{10} ties with the listed tree. Generated from `BGSpiderTableData.lean` by `paper/gen_table.py`.

n	maximizer	π/ρ^{n-1}	n	maximizer	π/ρ^{n-1}	n	maximizer	π/ρ^{n-1}
4	A_1	1.3452	167	$C^3 A_5^8 A_4^8$	1.1378	330	$A_5^{25} A_4^6$	1.1288
5	C^2	1.3129	168	$C^3 A_5^{13} A_4^2$	1.1383	331	A_5^{30}	1.1304
6	$C A_1$	1.2904	169	$C^3 A_5^9 A_4^7$	1.1376	332	$A_5^{26} A_4^5$	1.1291
7	C^3	1.3029	170	$C^4 A_5^{13} A_4^2$	1.1380	333	$C A_5^{30}$	1.1283
8	$C^2 A_1$	1.2657	171	$C^3 A_5^{10} A_4^6$	1.1373	334	$A_5^{27} A_4^4$	1.1293
9	C^4	1.2929	172	$C^4 A_5^{14} A_4$	1.1378	335	$A_5^{23} A_4^9$	1.1277
10	$C^2 A_2$	1.2658	173	$C^3 A_5^{11} A_4^5$	1.1371	336	$A_5^{28} A_4^3$	1.1296
11	C^5	1.2829	174	$C^4 A_5^{15}$	1.1375	337	$A_5^{24} A_4^8$	1.1280

n	maximizer	π/ρ^{n-1}	n	maximizer	π/ρ^{n-1}	n	maximizer	π/ρ^{n-1}
12	$C^2 A_3$	1.2609	175	$C^3 A_5^{12} A_4^4$	1.1369	338	$A_5^{29} A_4^2$	1.1299
13	C^6	1.2731	176	$C^5 A_5^{15}$	1.1365	339	$A_5^{25} A_4^7$	1.1283
14	$C^3 A_3$	1.2620	177	$C^3 A_5^{13} A_4^3$	1.1366	340	$A_5^{30} A_4$	1.1302
15	C^7	1.2633	178	$C^2 A_5^{10} A_4^7$	1.1359	341	$A_5^{26} A_4^6$	1.1286
16	$C^3 A_4$	1.2587	179	$C^3 A_5^{14} A_4^2$	1.1364	342	A_5^{31}	1.1304
17	C^8	1.2536	180	$C^2 A_5^{11} A_4^6$	1.1357	343	$A_5^{27} A_4^5$	1.1289
18	$C^4 A_4$	1.2575	181	$C^3 A_5^{15} A_4$	1.1361	344	$A_6 A_5^{30}$	1.1281
19	C^9	1.2440	182	$C^2 A_5^{12} A_4^5$	1.1354	345	$A_5^{28} A_4^4$	1.1292
20	$C^4 A_5$	1.2535	183	$C^3 A_5^{16}$	1.1359	346	$A_5^{24} A_4^9$	1.1275
21	$C^3 A_5^2$	1.2344	184	$C^3 A_5^{12} A_4^5$	1.1352	347	$A_5^{29} A_4^3$	1.1295
22	$C^5 A_5$	1.2504	185	$C^4 A_5^{16}$	1.1355	348	$A_5^{25} A_4^8$	1.1278
23	$C^4 A_5^2$	1.2335	186	$C^3 A_5^{13} A_4^4$	1.1350	349	$A_5^{30} A_4^2$	1.1298
24	$C^5 A_6$	1.2454	187	$C^2 A_5^{10} A_4^8$	1.1346	350	$A_5^{26} A_4^7$	1.1281
25	$C^4 A_4 A_3$	1.2336	188	$C^3 A_5^{14} A_4^3$	1.1349	351	$A_5^{31} A_4$	1.1301
26	$C^6 A_6$	1.2409	189	$C^2 A_5^{11} A_4^7$	1.1344	352	$A_5^{27} A_4^6$	1.1285
27	$C^4 A_4^2$	1.2336	190	$C^3 A_5^{15} A_4^2$	1.1347	353	A_5^{32}	1.1304
28	$C^6 A_7$	1.2351	191	$C^2 A_5^{12} A_4^6$	1.1342	354	$A_5^{28} A_4^5$	1.1288
29	$C^5 A_4^2$	1.2322	192	$C^3 A_5^{16} A_4$	1.1345	355	$A_6 A_5^{31}$	1.1281
30	$C^7 A_7$	1.2296	193	$C^2 A_5^{13} A_4^5$	1.1340	356	$A_5^{29} A_4^4$	1.1291
31	$C^5 A_5 A_4$	1.2297	194	$C^3 A_5^{17}$	1.1343	357	$A_5^{25} A_4^9$	1.1272
32	$C^7 A_8$	1.2232	195	$C^2 A_5^{14} A_4^4$	1.1339	358	$A_5^{30} A_4^3$	1.1294
33	$C^5 A_5^2$	1.2271	196	$C^4 A_5^{17}$	1.1336	359	$A_5^{26} A_4^8$	1.1276
34	$C^8 A_8$	1.2171	197	$C^2 A_5^{15} A_4^3$	1.1337	360	$A_5^{31} A_4^2$	1.1298
35	$C^6 A_5^2$	1.2246	198	$C A_5^{12} A_4^7$	1.1332	361	$A_5^{27} A_4^7$	1.1280
36	$C^4 A_4^3$	1.2162	199	$C^2 A_5^{16} A_4^2$	1.1335	362	$A_5^{32} A_4$	1.1301
37	$C^7 A_5^2$	1.2206	200	$C^2 A_5^{12} A_4^7$	1.1330	363	$A_5^{28} A_4^6$	1.1283
38	$C^5 A_4^3$	1.2159	201	$C^2 A_5^{17} A_4$	1.1333	364	A_5^{33}	1.1304
39	$C^7 A_6 A_5$	1.2167	202	$C^2 A_5^{13} A_4^6$	1.1329	365	$A_5^{29} A_4^5$	1.1287
40	$C^5 A_5 A_4^2$	1.2138	203	$C^2 A_5^{18}$	1.1331	366	$A_6 A_5^{32}$	1.1281
41	$C^7 A_6^2$	1.2128	204	$C^2 A_5^{14} A_4^5$	1.1328	367	$A_5^{30} A_4^4$	1.1290
42	$C^6 A_5 A_4^2$	1.2118	205	$C^3 A_5^{18}$	1.1329	368	$A_5^{26} A_4^9$	1.1270
43	$C^8 A_6^2$	1.2082	206	$C^2 A_5^{15} A_4^4$	1.1327	369	$A_5^{31} A_4^3$	1.1294
44	$C^6 A_5^2 A_4$	1.2102	207	$C A_5^{12} A_4^8$	1.1324	370	$A_5^{27} A_4^8$	1.1274
45	$C^8 A_7 A_6$	1.2031	208	$C^2 A_5^{16} A_4^3$	1.1325	371	$A_5^{32} A_4^2$	1.1297
46	$C^6 A_5^3$	1.2085	209	$C A_5^{13} A_4^7$	1.1323	372	$A_5^{28} A_4^7$	1.1278
47	$C^5 A_4^4$	1.2030	210	$C^2 A_5^{17} A_4^2$	1.1324	373	$A_5^{33} A_4$	1.1301
48	$C^7 A_5^3$	1.2056	211	$C A_5^{14} A_4^6$	1.1321	374	$A_5^{29} A_4^6$	1.1282
49	$C^5 A_5 A_4^3$	1.2013	212	$C^2 A_5^{18} A_4$	1.1323	375	A_5^{34}	1.1304
50	$C^7 A_6 A_5^2$	1.2020	213	$C A_5^{15} A_4^5$	1.1320	376	$A_5^{30} A_4^5$	1.1286
51	$C^6 A_5 A_4^3$	1.1998	214	$C^2 A_5^{19}$	1.1322	377	$A_6 A_5^{33}$	1.1282
52	$C^7 A_6^2 A_5$	1.1983	215	$C A_5^{16} A_4^4$	1.1319	378	$A_5^{31} A_4^4$	1.1289
53	$C^6 A_5^2 A_4^2$	1.1984	216	$C^3 A_5^{19}$	1.1316	379	$A_5^{27} A_4^9$	1.1268
54	$C^7 A_6^3$	1.1947	217	$C A_5^{17} A_4^3$	1.1317	380	$A_5^{32} A_4^3$	1.1293
55	$C^6 A_5^3 A_4$	1.1970	218	$C A_5^{13} A_4^8$	1.1315	381	$A_5^{28} A_4^8$	1.1272
56	$C^5 A_4^5$	1.1924	219	$C A_5^{18} A_4^2$	1.1316	382	$A_5^{33} A_4^2$	1.1297
57	$C^6 A_5^4$	1.1956	220	$C A_5^{14} A_4^7$	1.1314	383	$A_5^{29} A_4^7$	1.1276
58	$C^5 A_5 A_4^4$	1.1910	221	$C A_5^{19} A_4$	1.1315	384	$A_5^{34} A_4$	1.1301
59	$C^7 A_5^4$	1.1934	222	$C A_5^{15} A_4^6$	1.1314	385	$A_5^{30} A_4^6$	1.1280
60	$C^6 A_5 A_4^4$	1.1897	223	$C^2 A_5^{19} A_4$	1.1314	386	A_5^{35}	1.1304
61	$C^8 A_5^4$	1.1900	224	$C A_5^{16} A_4^5$	1.1313	387	$A_5^{31} A_4^5$	1.1284
62	$C^6 A_5^2 A_4^3$	1.1885	225	$C^2 A_5^{20}$	1.1313	388	$A_6 A_5^{34}$	1.1282
63	$C^8 A_6 A_5^3$	1.1867	226	$C A_5^{17} A_4^4$	1.1312	389	$A_5^{32} A_4^4$	1.1288

n	maximizer	π/ρ^{n-1}	n	maximizer	π/ρ^{n-1}	n	maximizer	π/ρ^{n-1}
64	$C^6 A_5^3 A_4^2$	1.1873	227	$A_5^{14} A_4^8$	1.1311	390	$A_5^{28} A_4^9$	1.1267
65	$C^5 A_5^6 A_4$	1.1835	228	$C A_5^{18} A_4^3$	1.1311	391	$A_5^{33} A_4^3$	1.1292
66	$C^6 A_5^4 A_4$	1.1862	229	$A_5^{15} A_4^7$	1.1310	392	$A_5^{29} A_4^8$	1.1271
67	$C^5 A_5^5 A_4$	1.1824	230	$C A_5^{19} A_4^2$	1.1311	393	$A_5^{34} A_4^2$	1.1296
68	$C^6 A_5^2 A_4$	1.1850	231	$A_5^{16} A_4^6$	1.1309	394	$A_5^{30} A_4^4$	1.1275
69	$C^5 A_5^2 A_4$	1.1812	232	$C A_5^{20} A_4$	1.1310	395	$A_5^{35} A_4$	1.1300
70	$C^7 A_5^2 A_4$	1.1832	233	$A_5^{17} A_4^5$	1.1309	396	$A_5^{31} A_4^6$	1.1279
71	$C^6 A_5^2 A_4$	1.1801	234	$C A_5^{21} A_4$	1.1309	397	$A_5^{36} A_4$	1.1304
72	$C^8 A_5^2 A_4$	1.1802	235	$A_5^{18} A_4^4$	1.1308	398	$A_5^{32} A_4^5$	1.1283
73	$C^6 A_5^3 A_4$	1.1791	236	$A_5^{14} A_4^9$	1.1307	399	$A_6 A_5^{35}$	1.1282
74	$C^8 A_6 A_5^3$	1.1771	237	$A_5^{19} A_4^3$	1.1307	400	$A_5^{33} A_4^4$	1.1288
75	$C^6 A_5^4 A_4$	1.1782	238	$A_5^{15} A_4^8$	1.1307	401	$A_5^{29} A_4^9$	1.1265
76	$C^5 A_5^5 A_4$	1.1750	239	$A_5^{20} A_4^2$	1.1306	402	$A_5^{34} A_4^3$	1.1292
77	$C^6 A_5^5 A_4$	1.1772	240	$A_5^{16} A_4^7$	1.1306	403	$A_5^{30} A_4^8$	1.1269
78	$C^5 A_5^2 A_4$	1.1740	241	$C A_5^{20} A_4^2$	1.1306	404	$A_5^{35} A_4^2$	1.1296
79	$C^6 A_5^6 A_4$	1.1762	242	$A_5^{17} A_4^6$	1.1306	405	$A_5^{31} A_4^7$	1.1274
80	$C^5 A_5^3 A_4$	1.1730	243	$C A_5^{21} A_4$	1.1305	406	$A_5^{36} A_4$	1.1300
81	$C^7 A_5^6 A_4$	1.1746	244	$A_5^{18} A_4^5$	1.1306	407	$A_5^{32} A_4^6$	1.1278
82	$C^5 A_5^4 A_4$	1.1720	245	$C A_5^{22} A_4$	1.1305	408	$A_5^{37} A_4$	1.1304
83	$C^8 A_5^6 A_4$	1.1718	246	$A_5^{19} A_4^4$	1.1306	409	$A_5^{33} A_4^5$	1.1282
84	$C^6 A_5^4 A_4$	1.1712	247	$A_5^{15} A_4^9$	1.1302	410	$A_6 A_5^{36}$	1.1282
85	$C^8 A_6 A_5^5$	1.1688	248	$A_5^{20} A_4^3$	1.1305	411	$A_5^{34} A_4^4$	1.1287
86	$C^6 A_5^5 A_4$	1.1704	249	$A_5^{16} A_4^8$	1.1303	412	$A_5^{30} A_4^9$	1.1263
87	$C^5 A_5^2 A_4$	1.1678	250	$A_5^{21} A_4^2$	1.1305	413	$A_5^{35} A_4^3$	1.1291
88	$C^6 A_5^6 A_4$	1.1696	251	$A_5^{17} A_4^7$	1.1303	414	$A_5^{31} A_4^8$	1.1268
89	$C^5 A_5^3 A_4$	1.1669	252	$A_5^{22} A_4$	1.1305	415	$A_5^{36} A_4^2$	1.1296
90	$C^6 A_5^7 A_4$	1.1688	253	$A_5^{18} A_4^6$	1.1303	416	$A_5^{32} A_4^7$	1.1272
91	$C^5 A_5^4 A_4$	1.1661	254	$A_5^{23} A_4$	1.1304	417	$A_5^{37} A_4$	1.1300
92	$C^7 A_5^7 A_4$	1.1672	255	$A_5^{19} A_4^5$	1.1303	418	$A_5^{33} A_4^6$	1.1277
93	$C^5 A_5^5 A_4$	1.1653	256	$C A_5^{23} A_4$	1.1301	419	$A_5^{38} A_4$	1.1304
94	$C^8 A_5^7 A_4$	1.1646	257	$A_5^{20} A_4^4$	1.1304	420	$A_5^{34} A_4^5$	1.1282
95	$C^5 A_5^6 A_4$	1.1645	258	$A_5^{16} A_4^9$	1.1298	421	$A_6 A_5^{37}$	1.1282
96	$C^5 A_5^2 A_4$	1.1623	259	$A_5^{21} A_4^3$	1.1304	422	$A_5^{35} A_4^4$	1.1286
97	$C^6 A_5^6 A_4$	1.1637	260	$A_5^{17} A_4^8$	1.1299	423	$A_5^{31} A_4^9$	1.1262
98	$C^5 A_5^3 A_4$	1.1616	261	$A_5^{22} A_4^2$	1.1304	424	$A_5^{36} A_4^3$	1.1291
99	$C^6 A_5^7 A_4$	1.1631	262	$A_5^{18} A_4^7$	1.1300	425	$A_5^{32} A_4^8$	1.1266
100	$C^5 A_5^4 A_4$	1.1609	263	$A_5^{23} A_4$	1.1304	426	$A_5^{37} A_4^2$	1.1295
101	$C^6 A_5^8 A_4$	1.1624	264	$A_5^{19} A_4^6$	1.1300	427	$A_5^{33} A_4^7$	1.1271
102	$C^5 A_5^5 A_4$	1.1602	265	$A_5^{24} A_4$	1.1304	428	$A_5^{38} A_4$	1.1300
103	$C^7 A_5^8 A_4$	1.1608	266	$A_5^{20} A_4^5$	1.1301	429	$A_5^{34} A_4^6$	1.1276
104	$C^5 A_5^6 A_4$	1.1595	267	$C A_5^{24} A_4$	1.1298	430	$A_5^{39} A_4$	1.1304
105	$C^8 A_5^8 A_4$	1.1582	268	$A_5^{21} A_4^4$	1.1302	431	$A_5^{35} A_4^5$	1.1281
106	$C^5 A_5^7 A_4$	1.1588	269	$A_5^{17} A_4^9$	1.1295	432	$A_6 A_5^{38}$	1.1282
107	$C^5 A_5^3 A_4$	1.1568	270	$A_5^{22} A_4^3$	1.1302	433	$A_5^{36} A_4^4$	1.1285
108	$C^5 A_5^8 A_4$	1.1581	271	$A_5^{18} A_4^8$	1.1296	434	$A_6 A_5^{37}$	1.1260
109	$C^5 A_5^4 A_4$	1.1562	272	$A_5^{23} A_4^2$	1.1303	435	$A_5^{37} A_4^3$	1.1290
110	$C^5 A_5^9 A_4$	1.1574	273	$A_5^{19} A_4^7$	1.1297	436	$A_5^{33} A_4^8$	1.1265
111	$C^5 A_5^5 A_4$	1.1556	274	$A_5^{24} A_4$	1.1304	437	$A_5^{38} A_4^2$	1.1295
112	$C^6 A_5^9 A_4$	1.1568	275	$A_5^{20} A_4^6$	1.1298	438	$A_5^{34} A_4^7$	1.1270
113	$C^5 A_5^6 A_4$	1.1551	276	$A_5^{25} A_4$	1.1304	439	$A_5^{39} A_4$	1.1300
114	$C^7 A_5^9 A_4$	1.1552	277	$A_5^{21} A_4^5$	1.1299	440	$A_5^{35} A_4^6$	1.1275
115	$C^5 A_5^7 A_4$	1.1545	278	$C A_5^{25} A_4$	1.1295	441	$A_5^{40} A_4$	1.1304

n	maximizer	π/ρ^{n-1}	n	maximizer	π/ρ^{n-1}	n	maximizer	π/ρ^{n-1}
116	$C^4 A_5^4 A_4^7$	1.1527	279	$A_5^{22} A_4^4$	1.1300	442	$A_5^{36} A_4^5$	1.1280
117	$C^5 A_5^8 A_4^2$	1.1539	280	$A_5^{18} A_4^9$	1.1291	443	$A_6 A_5^{39}$	1.1282
118	$C^4 A_5^9 A_4^6$	1.1521	281	$A_5^{23} A_4^3$	1.1301	444	$A_5^{37} A_4^4$	1.1285
119	$C^5 A_5^9 A_4$	1.1534	282	$A_5^{19} A_4^8$	1.1293	445	$A_6^2 A_5^{38}$	1.1261
120	$C^4 A_5^6 A_4^5$	1.1515	283	$A_5^{24} A_4^2$	1.1302	446	$A_5^{38} A_4^3$	1.1290
121	$C^5 A_5^{10}$	1.1528	284	$A_5^{20} A_4^7$	1.1294	447	$A_5^{34} A_4^8$	1.1264
122	$C^5 A_5^6 A_4^5$	1.1510	285	$A_5^{25} A_4$	1.1303	448	$A_5^{39} A_4^2$	1.1295
123	$C^6 A_5^{10}$	1.1520	286	$A_5^{21} A_4^6$	1.1296	449	$A_5^{35} A_4^7$	1.1269
124	$C^5 A_5^7 A_4^4$	1.1506	287	A_5^{26}	1.1304	450	$A_5^{40} A_4$	1.1299
125	$C^7 A_5^{10}$	1.1502	288	$A_5^{22} A_4^5$	1.1297	451	$A_5^{36} A_4^6$	1.1274
126	$C^5 A_5^8 A_4^3$	1.1501	289	$C A_5^{26}$	1.1292	452	A_5^{41}	1.1304
127	$C^4 A_5^9 A_4^7$	1.1487	290	$A_5^{23} A_4^4$	1.1299	453	$A_5^{37} A_4^5$	1.1279
128	$C^5 A_5^9 A_4^2$	1.1496	291	$A_5^{19} A_4^9$	1.1288	454	$A_6 A_5^{40}$	1.1283
129	$C^4 A_5^6 A_4^6$	1.1482	292	$A_5^{24} A_4^3$	1.1300	455	$A_5^{38} A_4^4$	1.1284
130	$C^5 A_5^{10} A_4$	1.1492	293	$A_5^{20} A_4^8$	1.1290	456	$A_6^2 A_5^{39}$	1.1261
131	$C^4 A_5^7 A_4^5$	1.1477	294	$A_5^{25} A_4^2$	1.1301	457	$A_5^{39} A_4^3$	1.1289
132	$C^5 A_5^{11}$	1.1487	295	$A_5^{21} A_4^7$	1.1292	458	$A_5^{35} A_4^8$	1.1263
133	$C^4 A_5^8 A_4^4$	1.1472	296	$A_5^{26} A_4$	1.1303	459	$A_5^{40} A_4^2$	1.1294
134	$C^6 A_5^{11}$	1.1477	297	$A_5^{22} A_4^6$	1.1293	460	$A_5^{36} A_4^7$	1.1268
135	$C^4 A_5^9 A_4^3$	1.1468	298	A_5^{27}	1.1304	461	$A_5^{41} A_4$	1.1299
136	$C^7 A_5^{11}$	1.1458	299	$A_5^{23} A_4^5$	1.1295	462	$A_5^{37} A_4^6$	1.1273
137	$C^4 A_5^{10} A_4^2$	1.1463	300	$C A_5^{27}$	1.1289	463	A_5^{42}	1.1304
138	$C^4 A_5^6 A_4^7$	1.1451	301	$A_5^{24} A_4^4$	1.1297	464	$A_5^{38} A_4^5$	1.1278
139	$C^5 A_5^{10} A_4^2$	1.1459	302	$A_5^{20} A_4^9$	1.1285	465	$A_6 A_5^{41}$	1.1283
140	$C^4 A_5^7 A_4^6$	1.1447	303	$A_5^{25} A_4^3$	1.1299	466	$A_5^{39} A_4^4$	1.1283
141	$C^5 A_5^{11} A_4$	1.1455	304	$A_5^{21} A_4^8$	1.1287	467	$A_6^2 A_5^{40}$	1.1261
142	$C^4 A_5^8 A_4^5$	1.1444	305	$A_5^{26} A_4^2$	1.1301	468	$A_5^{40} A_4^3$	1.1289
143	$C^5 A_5^{12}$	1.1451	306	$A_5^{22} A_4^7$	1.1289	469	$A_5^{36} A_4^8$	1.1261
144	$C^4 A_5^9 A_4^4$	1.1440	307	$A_5^{27} A_4$	1.1303	470	$A_5^{41} A_4^2$	1.1294
145	$C^6 A_5^{12}$	1.1439	308	$A_5^{23} A_4^6$	1.1291	471	$A_5^{37} A_4^7$	1.1267
146	$C^4 A_5^{10} A_4^3$	1.1436	309	A_5^{28}	1.1304	472	$A_5^{42} A_4$	1.1299
147	$C^3 A_5^7 A_4^7$	1.1424	310	$A_5^{24} A_4^5$	1.1294	473	$A_5^{38} A_4^6$	1.1272
148	$C^4 A_5^{11} A_4^2$	1.1432	311	$C A_5^{28}$	1.1287	474	A_5^{43}	1.1304
149	$C^3 A_5^8 A_4^6$	1.1420	312	$A_5^{25} A_4^4$	1.1296	475	$A_5^{39} A_4^5$	1.1278
150	$C^4 A_5^{12} A_4$	1.1428	313	$A_5^{21} A_4^9$	1.1282	476	$A_6 A_5^{42}$	1.1283
151	$C^4 A_5^8 A_4^6$	1.1417	314	$A_5^{26} A_4^3$	1.1298	477	$A_5^{40} A_4^4$	1.1283
152	$C^4 A_5^{13}$	1.1424	315	$A_5^{22} A_4^8$	1.1285	478	$A_6^2 A_5^{41}$	1.1261
153	$C^4 A_5^9 A_4^5$	1.1414	316	$A_5^{27} A_4^2$	1.1300	479	$A_5^{41} A_4^3$	1.1288
154	$C^5 A_5^{13}$	1.1419	317	$A_5^{23} A_4^7$	1.1287	480	$A_5^{37} A_4^8$	1.1260
155	$C^4 A_5^{10} A_4^4$	1.1411	318	$A_5^{28} A_4$	1.1302	481	$A_5^{42} A_4^2$	1.1294
156	$C^6 A_5^{13}$	1.1405	319	$A_5^{24} A_4^6$	1.1290	482	$A_5^{38} A_4^7$	1.1266
157	$C^4 A_5^{11} A_4^3$	1.1408	320	A_5^{29}	1.1304	483	$A_5^{43} A_4$	1.1299
158	$C^3 A_5^8 A_4^7$	1.1399	321	$A_5^{25} A_4^5$	1.1292	484	$A_5^{39} A_4^6$	1.1271
159	$C^4 A_5^{12} A_4^2$	1.1405	322	$C A_5^{29}$	1.1285	485	A_5^{44}	1.1304
160	$C^3 A_5^9 A_4^6$	1.1395	323	$A_5^{26} A_4^4$	1.1294	486	$A_5^{40} A_4^5$	1.1277
161	$C^4 A_5^{13} A_4$	1.1401	324	$A_5^{22} A_4^9$	1.1279	487	$A_6 A_5^{43}$	1.1283
162	$C^3 A_5^{10} A_4^5$	1.1392	325	$A_5^{27} A_4^3$	1.1297	488	$A_5^{41} A_4^4$	1.1282
163	$C^4 A_5^{14}$	1.1398	326	$A_5^{23} A_4^8$	1.1282	489	$A_6^2 A_5^{42}$	1.1261
164	$C^3 A_5^{11} A_4^4$	1.1389	327	$A_5^{28} A_4^2$	1.1299	490	$A_5^{42} A_4^3$	1.1288
165	$C^5 A_5^{14}$	1.1391	328	$A_5^{24} A_4^7$	1.1285	491	$A_5^{38} A_4^8$	1.1259
166	$C^3 A_5^{12} A_4^3$	1.1386	329	$A_5^{29} A_4$	1.1302			